

Sesiunea 23

- Agenda
 - Rezolvare varianta BAC August 2021
 - Recapitulare pentru BAC

Rezolvare varianta BAC August 2021

Subiectul I

1. Rezolvare

- Aici trebuie sa tinem cont de procesul de aducere la acelasi numitor comun dar si de precedenta operatorilor.
- Intotdeauna, pentru a fi siguri de ce operator este luat in calcul mai intai, putem folosi paranteze pentru a delimita clar.
- Lucru ce se intampla si in varianta **d** care este cea corecta.
- Celelalte variante, deoarece nu folosesc paranteze, precedenta (ordinea) operatiilor este data strict de operatorii folositi si astfel nu mai obtinem raspunsul cerut in enunt.
- Raspuns corect: **D**

2. Rezolvare

- Subprogramul calculează suma produselor dintre paritatea cifrei ($n\%2$, care este 1 dacă cifra este impară și 0 dacă este pară) și valoarea cifrei ($n\%10$)

```
f(1234): Cifra este 4. 4 % 2 = 0. Rezultat: 0 * 4 + f(123) = 0 + f(123).
      f(123): Cifra este 3. 3 % 2 = 1. Rezultat: 1 * 3 + f(12) = 3 +
f(12).
      f(12): Cifra este 2. 2 % 2 = 0. Rezultat: 0 * 2 + f(1) = 0 +
f(1).
      f(1): Cifra este 1. 1 % 2 = 1. Rezultat: 1 * 1 + f(0) = 1 +
f(0).
      f(0): Condiția n!=0 este falsă, deci returnează 0.
```

- Acum calculam: $0 + 1 + 0 + 3 + 0$
- Raspuns corect: **B**

3. Rezolvare

cifrele se aleg în ordine crescătoare, iar o soluție se afișează imediat ce suma cifrelor devine **6** (chiar dacă se mai pot adăuga cifre).

Primele soluții sunt:

1023
1032
105
1203
123

1230

Deci a șasea soluție generată este: 1230

- Raspuns corect: A

4. Rezolvare:

Un arbore cu 10 noduri are întotdeauna: $(n-1)$ $10-1=9$ muchii și nu conține cicluri.

Dacă adăugăm o singură muchie între două noduri ale arborelui, se formează exact un ciclu elementar, deoarece între cele două noduri exista deja un drum unic în arbore.

Noua muchie închide acel drum și creează un singur ciclu.

- Raspuns corect: B

5. Rezolvare:

- Stim ca avem 21 de noduri: 1,2,3,4,5,..21
- Stim ca nodurile sunt grupate dupa ultima cifra:
 - Cira 1: 1, 11, 21 -> 3 noduri pentru care obtinem 3 muchii: [1,11], [1,21], [11,21]
 - Cifra 0: 10, 20 -> 2 noduri pentru care obtinem 1 muchie: [10,20]
 - Fiecare dintre cifrele 2,3,4,5,6,7,8,9 apar de cate 2 ori si astfel avem inca 8 grupe a cate 2 noduri adica 8 muchii
 - Astfel avem $3 + 1 + 8$ muchii = 12
 - Matricea de adiacenta are: $21^2 = 441$ elemente
 - Fiind graf neorientat fara bucle, numarul valorilor de 1 este $2m = 24$ deci numarul valorilor nule este $441 - 24 = 417$ adica: $21^2 - 2 * 12$
 - Ca si explicatie de ce apar $2m$ valori de 1 in matrice, daca exisata muchia [i,j] atunci avem 1 si pentru a[i][j] dar si pt a[j][i] deci fiecare muchie apare de 2 ori in matrice

- Raspuns corect: C

Subiectul II

- a

Algoritmul parcurge toate numerele de la 1 la 15 și afișează numerele care:

- se divid cu 3, dar nu cu 4
- SAU
- se divid cu 4, dar nu cu 3.

Verificăm fiecare număr:

- 3 → se divide cu 3, nu cu 4 → se afișează
- 4 → se divide cu 4, nu cu 3 → se afișează
- 6 → se divide cu 3, nu cu 4 → se afișează
- 8 → se divide cu 4, nu cu 3 → se afișează
- 9 → se divide cu 3, nu cu 4 → se afișează
- 12 → se divide și cu 3 și cu 4 → NU se afișează
- 15 → se divide cu 3, nu cu 4 → se afișează

Rezultatul final afișat este:

3 4 6 8 9 15

o b

Pentru ca algoritmul să afișeze valoarea 0, trebuie ca niciun număr de la 1 la n să nu respecte condiția:

- să se dividă cu x, dar nu cu y
SAU
- să se dividă cu y, dar nu cu x.

Asta se întâmplă atunci când numerele divizibile cu x sunt exact aceleași cu cele divizibile cu y.

Exemplul 1:

n = 5, x = 2, y = 2

Numerele divizibile cu 2 sunt aceleași pentru x și y.
Nu există niciun număr care să se dividă doar cu unul dintre ele.

Se afișează:

0

Si alt set similar ar fi n = 13, x = 3, y = 3

o c

```
#include<iostream>
using namespace std;
int main()
{
    int n,x,y;
    cin >> n >> x >> y;
    int ok = 0;
    for (int i = 1; i <= n; i++) {
        if (i % x == 0 && i % y != 0 ||
```

```

        i % x != 0 && i % y == 0
    ) {
        cout << i << " ";
        ok = 1;
    }
}
if (ok == 0) {
    cout << 0;
}
}

```

◦ d

```

citește n,x,y
(numere naturale nenule, x≤n, y≤n)
ok< - 0
i <- 1
┌cat timp i <= n execută
│┌dacă (i%x=0 și i%y≠0) sau
││ (i%x≠0 și i%y=0) atunci
││ scrie i, ' '
││ ok< - 1
│└─
│ i <- i + 1
│└─
└dacă ok=0 atunci scrie 0
└─

```

2. Rezolvare

```

strcpy(s,"ROMANIA");
i=strlen(s)-1;
for(j=3;j>=0;j--)
{
    aux=s[i];
    s[i]=s[i-j];
    s[i-j]=aux;
    i=i-j;
}
- Aici trebuie sa fim atenti pas cu pas ce litera este la pozitia
ceruta:
Inițial:

s = "ROMANIA"
Lungimea șirului este 7, deci:
i = strlen(s)-1 = 6
---

j = 3

```

```

Se interșchimbă s[6] cu s[3]:
A ⇔ A
Șirul rămâne:
ROMANIA
i = 6-3 = 3
---

j = 2
Se interșchimbă s[3] cu s[1]:
A ⇔ O
Șirul devine:
RAMONIA
i = 3-2 = 1
---

j = 1
Se interșchimbă s[1] cu s[0]:
A ⇔ R
Șirul devine:
ARMONIA
i = 1-1 = 0
---

j = 0
Se interșchimbă s[0] cu s[0], deci șirul nu se modifică.
Rezultatul final:
ARMONIA

```

- La final va conține "ARMONIA"

3. Rezolvare

```

#include<iostream>
#include <cstring>
using namespace std;

struct date {
    char CNP[14];
    int anNastere;
};

struct angajat {
    struct date dp;
    int venit;
}a[30];

int main()
{
    //La bac scrii doar ce e între --
    //-----
    a[0].dp.anNastere = 2000;

```

```
a[0].venit=4000;  
//-----  
}
```

Subiectul III

1. Rezolvare

```
#include<iostream>  
using namespace std;  
  
void cuburi(int n);  
  
int main()  
{  
    cuburi(5);  
    cout << endl;  
    cout << endl;  
    cuburi(7);  
}  
  
void cuburi(int n) {  
    for(int i = n; i >= 1; i--)  
    {  
        cout << i * i * i << " ";  
    }  
}
```

2. Rezolvare

```
#include <iostream>  
#include <cstring>  
  
using namespace std;  
  
int main() {  
  
    int n, k;  
    cin >> k >> n;  
    int matrice[n][n];  
    for (int i = 0; i < n; i++) {  
        for (int j = 0; j < n; j++) {  
            if (i == j) {  
                matrice[i][j] = k * (i+1);  
            } else if (j > i) {  
                matrice[i][j] = (k * (i+1)) + (j-i);  
            } else {  
                matrice[i][j] = (k * (i+1)) - (i-j);  
            }  
        }  
    }  
}
```

```

    }
}

for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        cout << matrice[i][j] << " ";
    }
    cout << endl;
}

return 0;
}

```

◦ Explicatie:

- Observam ca pe diagonala principala avem multiplu de k
- In dreapta diagonalei principale, valoarea este egala cu ce avem pe diagonala principala pe acea linie + distanta de la diagonala principala pana la coloana respectiva
- In stanga diagonalei principale, avem in oglinda ce avem in dreapta, adica ce e pe diagonala principala dar minus distanta de la diagonala principala pana la coloana respectiva

3. ◦ a

Vom construi un algoritm prin care vom face urmatoarele:

1. Normalizăm ultimele două cifre

Pentru un număr x:

luăm ultimele două cifre:

a = (x / 10) % 10

b = x % 10

le ordonăm:

dacă a > b, le inversăm

Astfel:

12 devine (1,2)

21 devine (1,2)

20 devine (0,2)

02 devine (0,2)

2. Construim frecvențe pentru A

Pentru fiecare număr din A:

calculăm perechea (a, b) normalizată

incrementăm frecvența acestei perechi

3. Calculăm răspunsul folosind B

Pentru fiecare număr din B:

calculăm aceeași pereche normalizată
adunăm frecvența corespunzătoare din A

Algoritmul este eficient din punct de vedere al timpului de execuție deoarece avem o complexitate $O(n_a + n_b)$ deci $O(n)$.

o b

```
#include <iostream>
#include <fstream>
using namespace std;

int main()
{
    ifstream fin("numere.in");

    int na, nb;
    fin >> na >> nb;

    long long fr[100] = {0};
    long long total = 0;

    // A
    for(int i = 0; i < na; i++)
    {
        int x;
        fin >> x;

        int a = (x / 10) % 10;
        int b = x % 10;

        if(a > b) swap(a, b);

        fr[a * 10 + b]++;
    }

    // B
    for(int i = 0; i < nb; i++)
    {
        int y;
        fin >> y;

        int a = (y / 10) % 10;
        int b = y % 10;

        if(a > b) swap(a, b);

        total += fr[a * 10 + b];
    }

    cout << total;
```

```
    return 0;  
}
```